

A
Mini Project Report
On
Event Recommendation System
Submitted to CMR ENGINEERING COLLEGE
In Partial Fulfillment of the requirements for the Award of Degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted
By

D. Gnanpreeth Reddy	(228R1A05D9)
G. Srikanth	(228R1A05E3)
L.Aravind	(228R1A05F6)
S.Bhavya	(228R1A05H7)

Under the Esteemed guidance of
Mr. D. Uma Vishveshwar
Assistant Professor, Department of CSE



Department of Computer Science & Engineering
CMR ENGINEERING COLLEGE
UGC AUTONOMOUS

(Approved by AICTE, NEW DELHI, Affiliated to JNTU, Hyderabad)
Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)

2024-2025

CMR ENGINEERING COLLEGE

UGC AUTONOMOUS

(Accredited by NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)

Kandlakoya, Medchal Road, Hyderabad-501 401

Department of Computer Science & Engineering



CERTIFICATE

This is to certify that the project entitled “**Event Recommendation System**” is a bonafide work carried out by

D. Gnanpreeth Reddy	(228R1A05D9)
G. Srikanth	(228R1A05E3)
L.Aravind	(228R1A05F6)
S.Bhavya	(228R1A05H7)

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this Mini project have been verified and are found to be satisfactory. The results embodied in this Mini project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide	Mini Project Coordinator	Head of the Department
Mr. D.Uma Vishveshwar Assistant Professor Department of CSE, CMREC, Hyderabad	Mr. S Kiran Kumar Assistant Professor Department of CSE, CMREC, Hyderabad	Dr. Sheo Kumar Professor & H.O.D Department of CSE, CMREC, Hyderabad

External Examiner

DECLARATION

This is to certify that the work reported in the present Mini project entitled “**Event Recommendation System**” is a record of bonafide work done by us in the Department of Computer Science and Engineering, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

D. Gnanpreeth Reddy	(228R1A05D9)
G. Srikanth	(228R1A05E3)
L.Aravind	(228R1A05F6)
S.Bhavya	(228R1A05H7)

ACKNOWLEDGMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Sheo Kumar**, HOD, **Department of CSE, CMR Engineering College** for their constant support.

We are extremely thankful to **Mr. D. Uma vishveshwar**, Assistant Professor, Internal Guide, Department of CSE, for his/her constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We thank **Mr. S. Kiran Kumar**, Assistant Professor, CSE Department, Mini Project Coordinator for his/her constant support in carrying out the project activities and reviews.

We express my thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided me for every step.

D. Gnanpreeth Reddy	(228R1A05D9)
G. Srikanth	(228R1A05E3)
L. Aravind	(228R1A05F6)
S.Bhavya	(228R1A05H7)

CONTENTS

TOPIC	PAGENO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	ii
1. INTRODUCTION	
1.1 Introduction and Objectives	01
1.2 Purpose of the Project	02
1.3 Existing System and Disadvantages	03
1.4 Proposed System and Advantages	04
2. LITERATURE SURVEY	06
3. SOFTWARE REQUIREMENT ANALYSIS	
3.1 Problem Specification	08
3.2 Modules	08
3.3 Functional Requirements	10
3.4 Non-Functional Requirements	11
3.5 Feasibility Study	11
4. SOFTWARE AND HARDWARE REQUIREMENTS	
4.1 Hardware Configuration	17
4.2 Software Configuration	17
5. SOFTWARE DESIGN	
5.1 Data Flow Diagrams	18
5.2 UML Diagrams	20
5.3 System Architecture	26
6. CODING AND IMPLEMENTATION	
6.1 Methodology	28
6.1 Sample Code	29
6.2 Data Dictionary	44
7. TESTING	
7.1 Introduction of Testing	48
7.2 Unit Testing	49
7.3 Integration Testing	49

8. OUTPUT SCREENS	50
9. CONCLUSION	53
10. FUTURE ENHANCEMENTS	54
11. REFERENCES	
11.1 Text Books	55
11.2 Websites	55

ABSTRACT

ABSTRACT

With the rapid growth of online platforms and digital events, users are often overwhelmed by the abundance of event choices. This has led to the need for intelligent recommendation systems that can filter and personalize event suggestions based on user-specific data. An Event Recommendation System is designed to analyze various inputs such as user preferences, historical behavior, location, social connections, and real-time context to provide relevant and engaging event recommendations.

The proposed system combines collaborative filtering, content-based filtering, and hybrid recommendation strategies to improve accuracy and relevance. Collaborative filtering identifies user similarities and suggests events based on peer preferences, while content-based filtering focuses on matching events to individual user profiles. A hybrid approach merges these techniques to address limitations such as cold start and data sparsity problems.

The system architecture includes data preprocessing, feature extraction, similarity computation, and ranking algorithms. Machine learning models are employed to continuously refine recommendations using user feedback and interaction logs. Moreover, the system can adapt to changing user interests over time by incorporating dynamic learning mechanisms.

Evaluation metrics such as precision, recall, and user satisfaction scores demonstrate the system's effectiveness in enhancing user engagement and attendance rates. The Event Recommendation System not only benefits end-users by saving time and improving event discovery but also supports event organizers in targeting the right audience more efficiently. This solution holds potential for deployment in social media platforms, event management apps, and digital marketing tools.

LIST OF FIGURES

S. NO.	FIG. NO.	DESCRIPTION	PAGE NO.
1	5.1.1	Class Diagram	18
2	5.2.1	Use Case Diagram	20
3	5.2.2	Sequence Diagram	22
4	5.2.3	Activity Diagram	24
5	5.3.1	System Architecture	26
6	8.a	Main Page	50
7	8.b	Concerts recommendation Page	50
8	8.c	Sports recommendation Page	51
9	8.d	Festival recommendation Page	51
10	8.e	Contact Page	52

LIST OF TABLES

S. NO.	DESCRIPTION	PAGE NO.
1	Literature Survey	06
2	System Testing	48
3	Unit Testing	49
4	Integration Testing	49

CHAPTER - 1

INTRODUCTION

1. INTRODUCTION

1.1 INTRODUCTION AND OBJECTIVES

In the modern digital landscape, the sheer volume of available information can overwhelm users, especially when it comes to discovering and selecting events of interest. Events—ranging from concerts, exhibitions, webinars, and workshops to conferences and community meetups—are being hosted daily across various platforms. As users browse through online platforms and social media, they are constantly exposed to numerous events, most of which may not align with their preferences, schedules, or locations. This unstructured and vast flow of event data makes it challenging for individuals to identify relevant events efficiently. Consequently, users often miss out on opportunities that genuinely match their interests, while event organizers struggle to reach the right target audience.

To overcome these challenges, the development of an intelligent Event Recommendation System becomes essential. Such a system can bridge the gap between users and event organizers by leveraging advanced computational methods to recommend events tailored to individual users' interests. The core idea behind this system is personalization—delivering content that is most relevant to the user. By analyzing historical data, user profiles, geographical information, time constraints, and social connections, the system can generate meaningful and context-aware event suggestions. This not only enhances user satisfaction but also boosts engagement and participation rates.

The Event Recommendation System integrates a variety of recommendation techniques to improve its performance. Collaborative filtering is used to find users with similar preferences and recommend events that like-minded individuals have attended or shown interest in. Content-based filtering, on the other hand, recommends events based on the characteristics of previously attended or liked events. To overcome the limitations of both methods, such as cold-start problems or lack of data, a hybrid recommendation model is employed. This combined approach increases the accuracy and reliability of the recommendations.

An important component of the system is its ability to handle dynamic user behavior. User preferences can evolve over time due to changing interests, trends, or life circumstances. Therefore, the system must be capable of learning and adapting continuously through feedback mechanisms. Real-time data, such as user interactions, event ratings, or clicks, can be used to refine and update recommendations. By incorporating such adaptive learning, the system remains effective and relevant over extended periods.

The objectives of developing this Event Recommendation System are multifaceted. Firstly, it aims to assist users in discovering relevant and timely events without the need for exhaustive searching. Secondly, it seeks to improve the efficiency of event marketing by helping organizers target users who are more likely to attend

their events. Thirdly, it enhances the overall user experience by reducing information overload and presenting only high-quality, personalized suggestions. Additionally, the system is designed to be scalable, supporting large datasets and handling a growing user base without performance degradation.

Furthermore, the system aims to implement a clean and intuitive user interface that encourages user interaction and engagement. The feedback collected through the interface—such as likes, dislikes, bookmarks, or event attendance—can be utilized to further refine the recommendation process. The system also considers contextual factors such as time of day, event proximity, and user availability to ensure that the recommendations are practical and actionable.

Another key objective is to handle new users and new events effectively. This is achieved through content-based techniques that rely on available metadata such as event category, location, and description. The system is also expected to provide explainable recommendations so users understand why a particular event was suggested, which can increase trust and system adoption.

Ultimately, the Event Recommendation System provides mutual benefits for both users and event organizers. Users receive a personalized and convenient way to explore events that matter to them, while organizers gain a powerful tool to boost visibility and attract a relevant audience. This system can be implemented across various platforms, including mobile applications, websites, and integrated social networks, making it a versatile solution for modern event discovery.

1.2 PURPOSE OF THE PROJECT

The main purpose of the Event Recommendation System project is to provide users with personalized and relevant event suggestions, helping them discover events that match their interests, preferences, and availability. In the digital age, users are bombarded with information from various sources, making it difficult to filter out meaningful and engaging events. This system aims to simplify and enhance the event discovery process by using intelligent recommendation algorithms.

The project is designed to analyze user-specific data such as event history, categories of interest, location, and engagement patterns to deliver curated recommendations. It uses techniques like collaborative filtering, content-based filtering, and hybrid models to improve recommendation accuracy and overcome limitations such as the cold-start problem. These models ensure that both new and returning users receive relevant suggestions, even with limited historical data.

One of the key purposes of this system is to save users time and effort in searching for events. Rather than browsing through hundreds of listings, users are shown events that are most likely to align with their tastes

and preferences. By offering a more efficient and engaging way to explore events, the system increases user satisfaction and encourages greater participation.

Another important goal of the project is to support event organizers in promoting their events to the right audience. By connecting events with interested users, the system helps improve attendance, boost visibility, and enhance event success. It provides a win-win situation for both users and organizers.

Additionally, the project seeks to build a scalable and adaptive system that evolves based on user feedback and behavior. As users interact with the platform—by attending events, liking suggestions, or providing ratings—the system learns and refines its recommendations, ensuring continuous improvement over time.

The system also considers contextual factors such as time, date, location, and event type, making the recommendations more practical and personalized. The purpose is not just to suggest events, but to do so in a way that feels intuitive, timely, and useful for the end user.

Ultimately, the Event Recommendation System aims to bridge the gap between users and the vast world of events around them, offering a smarter, more connected, and enjoyable experience for everyone involved.

1.3 EXISTING SYSTEM AND DISADVANTAGES

In current scenarios, most event discovery platforms such as social media networks (e.g., Facebook Events), event management websites (e.g., Eventbrite, Meetup), and calendar applications provide users with event suggestions based on generic criteria. These systems often rely on keyword searches, trending topics, or location-based filtering. While some of these platforms offer basic personalization features, such as recommending events based on previously attended or bookmarked events, the recommendation capabilities are often limited in scope and depth.

Many existing systems lack advanced data processing and personalization features. They often use simple rule-based algorithms or manual filtering methods, which do not effectively capture user preferences or adapt to evolving interests. In some cases, the recommendations are based solely on popular events or user demographics, ignoring individual behavior, feedback, or contextual relevance like time availability or proximity.

1. **Static Travel Planning Systems** – Traditional mapping and navigation applications (e.g., Google Maps, Apple Maps) provide static route recommendations based on distance and estimated travel time but lack deep personalization beyond basic options like toll-free routes or fastest paths.

2. **Tourism-Based Recommendation Systems** – Platforms like TripAdvisor and Expedia suggest destinations and attractions based on user reviews and general popularity, but they do not dynamically adjust recommendations based on real-time conditions.
3. **Collaborative Filtering-Based Systems** – Some applications use collaborative filtering (e.g., recommending places visited by similar users), but these systems often rely on historical data and lack real-time adaptability.
4. **Content-Based Filtering Systems** – Travel applications that recommend locations based on user preferences (e.g., "beach destinations" for a user interested in seaside travel) often fail to account for external factors like traffic or weather.

Disadvantages:

- **Lack of Personalization:** – Most travel recommendation systems provide generic suggestions and do not fully consider individual preferences like budget, interests, or travel history.
- **Limited Use of Historical Data:**– Existing platforms often fail to analyze user event history effectively, leading to repetitive or irrelevant recommendations.
- **Cold Start Problem:** –New users receive poor recommendations initially due to the absence of interaction history.
- **Static Recommendations:** Many systems do not adapt dynamically to user feedback or behavior over time.
- **No Context Awareness:** Recommendations are often made without considering real-time factors such as time, location, or user availability.

1.4 PROPOSED SYSTEM AND ADVANTAGES

The proposed Event Recommendation System aims to overcome the limitations of existing platforms by leveraging intelligent recommendation algorithms that provide personalized and relevant event suggestions to users. This system analyzes a wide range of user-specific data, including preferences, location, past behavior, interests, and interaction history, to deliver accurate and context-aware recommendations.

The system combines multiple recommendation techniques—such as **collaborative filtering**, **content-based filtering**, and **hybrid models**—to ensure both personalization and scalability. It also integrates **location-based services** and **temporal filtering** to suggest events that are not only relevant but also conveniently accessible in terms of time and distance.

An important feature of the proposed system is its **adaptive learning mechanism**. As users interact with the platform (e.g., by liking, attending, or dismissing events), the system continuously refines its suggestions using machine learning models. Additionally, a **feedback loop** allows users to rate recommendations, which helps further improve future suggestions. The system will also include a **user-friendly interface** for seamless event discovery and interaction.

Advantages:

- **High Personalization:**— Offers event suggestions tailored to each user's preferences, interests, and behavior.
- **Dynamic Learning:** – Continuously updates and improves recommendations based on user interactions and feedback.
- **Context-Aware Recommendations:** Considers time, location, and availability, providing relevant and timely suggestions.
- **Cold Start Handling:** Uses content-based filtering to provide quality recommendations even for new users with limited data.
- **Improved User Experience:** Simplifies event discovery and reduces the time and effort needed to find suitable events.
- **Feedback Integration:** Incorporates user feedback to continually enhance the quality of recommendations.

CHAPTER - 2

LITERATURE SURVEY

2. LITERATURE SURVEY

Ref no.	Research Work/Paper	Author / Year	Techniques	Experiments/ Observations	Remarks
[1]	Temporal Collaborative Filtering for Event Recommendation.	Xiong et al (2017)	Collaborative Filtering, Temporal Modelling, Recurrent Neural Networks	Incorporating temporal dynamics improves recommendation accuracy.	Time-aware models can capture changing user preferences.
[2]	Event Recommendation using Collaborative Filtering and Event Embedding's	Li et al (2019)	Collaborative Filtering, Event embedding's, Matrix Factorization	Combining collaborative filtering with event embeddings improves recommendation accuracy.	This approach leverages both user behaviour and event attributes.
[3]	Deep Collaborative Filtering for Event Recommendation on using Convolutional Neural Networks	Li et al. (2020)	Collaborative Filtering, Convolutional Neural Networks, Deep Recommendation	Deep learning-based collaborative filtering improves recommendation accuracy.	This approach enables learning complex patterns in user event interactions.
[4]	Scalable Collaborative Filtering for Event Recommendation using Distributed Tensor Factorization	Kim et al. (2018)	Collaborative Filtering, Distributed Tensor Factorization, Scalable.	Distributed tensor factorization enables scalable collaborative filtering.	This approach facilitates largescale event recommendation systems.

Collaborative filtering has become a core technique in developing effective event recommendation systems, and recent research has focused on enhancing its accuracy and scalability through various advanced methods. Xiong et al. (2017) introduced a model that incorporates temporal dynamics using recurrent neural networks, demonstrating that time-aware systems can capture evolving user preferences and significantly improve recommendation accuracy. Building on this, Li et al. (2019)

combined collaborative filtering with event embeddings and matrix factorization, showing that leveraging both user behavior and event-specific attributes leads to more precise recommendations. In a further development, Li et al. (2020) employed deep learning techniques, particularly convolutional neural networks, to enable the system to learn complex interaction patterns between users and events, resulting in better predictive performance. Finally, Kim et al. (2018) addressed the challenge of scalability by applying distributed tensor factorization, which allows for efficient handling of large-scale datasets and supports the deployment of scalable event recommendation systems. Collectively, these studies illustrate that integrating collaborative filtering with temporal modeling, deep learning, and tensor-based approaches enhances both the adaptability and performance of event recommendation systems.

CHAPTER - 3

SOFTWARE REQUIREMENT ANALYSIS

3. SOFTWARE REQUIREMENT ANALYSIS

3.1 PROBLEM SPECIFICATION

In today's digital era, the number of events being organized and promoted online is rapidly increasing across various platforms such as websites, social media, and mobile applications. These events range from concerts, webinars, workshops, and conferences to local community meetups. However, due to the overwhelming number of event listings and the lack of personalized filtering, users often face difficulty in discovering events that are truly relevant to their interests, schedules, or locations. As a result, many valuable events go unnoticed by potential attendees, and users miss opportunities for engagement, learning, or networking.

Traditional event listing platforms primarily display popular or trending events, with minimal personalization. They often rely on keyword searches and category filters, which may not be sufficient for effective event discovery. Moreover, users have to manually browse through long lists of events, leading to information overload and poor user experience. Additionally, event organizers struggle to reach their target audience effectively, resulting in low event turnout despite high-quality content.

The absence of intelligent recommendation features, context-aware suggestions, and real-time personalization in existing systems highlights a clear gap. Therefore, there is a need for a smart and efficient solution that can automatically analyze user behavior, preferences, location, and time availability to recommend the most suitable events.

The proposed Event Recommendation System addresses this issue by implementing advanced recommendation techniques—such as collaborative filtering, content-based filtering, and hybrid models—to deliver personalized event suggestions. The system also incorporates contextual data (like location and time) and user feedback to continuously refine recommendations. This not only enhances user satisfaction and engagement but also supports event organizers in connecting with the right audience.

3.2 MODULES

Here are the main modules in the *Event Recommendation System* and their functionalities:

1. User Management Module

Functionality:

Allows users to register, log in, and log out securely.

Manages user profiles including interests, preferences, and location.

Enables password recovery and account settings.

2. Event Management Module

Functionality:

Allows event organizers or admins to create, update, and delete event listings.

Stores event information such as title, description, category, date, time, location, and organizer details.

Supports event image uploads and RSVP limits.

3. Recommendation Engine Module

Functionality:

Provides personalized event suggestions using recommendation algorithms:

Content-Based Filtering: Based on user preferences and event attributes.

Collaborative Filtering: Based on similar users' event interests.

Hybrid Approach: Combines both for better accuracy.

Continuously updates recommendations based on user behavior and feedback.

4. Search and Filter Module

Functionality:

Allows users to search for events using keywords.

Provides filtering options based on categories, location, date, popularity, etc.

5. Feedback and Rating Module

Functionality:

Allows users to rate and review events they attended or are interested in.

Captures likes, dislikes, and feedback to improve future recommendations.

Displays average event ratings for other users.

3.3 FUNCTIONAL REQUIREMENTS

These describe the essential functions and features that the system must provide:

- **User Registration and Login**

Users must be able to create an account and securely log in.

- **User Profile Management**

Users can create and update personal preferences, interests, and location information.

- **Event Listing and Search**

Users can browse and search for events based on keywords, categories, or filters.

- **Personalized Recommendations**

The system recommends events based on user preferences, history, and behavior.

- **Feedback System**

Users can like, dislike, or rate events, which helps the system improve future recommendations.

- **Location-Based Filtering**

The system filters events based on the user's current or specified location.

- **Event Details View**

Users can view detailed information about events, including date, time, venue, and description.

- **Event Bookmarking or RSVP**

Users can save/bookmark events or mark them as "interested" or "going".

- **Admin Event Management**

Admins can create, edit, and delete events in the system.

3.4 NON-FUNCTIONAL REQUIREMENTS

These define system performance and quality attributes:

- **Usability**

The system must provide an intuitive and user-friendly interface.

- **Scalability**

It should support a growing number of users and events without performance degradation.

- **Performance**

Recommendations should be generated quickly (e.g., within 2–3 seconds).

- **Reliability**

The system should have high availability and minimal downtime.

- **Security**

User data and credentials must be securely stored and protected using encryption and secure protocols.

- **Maintainability**

The codebase should be modular and well-documented for easy updates and bug fixes.

3.5 FEASIBILITY STUDY

A feasibility study is conducted to evaluate whether the proposed Event Recommendation System is viable and practical from various perspectives. It ensures that the project is worth pursuing and can be developed successfully within the given constraints.

1. Technical Feasibility

The technical feasibility assesses whether the current technology stack, tools, and infrastructure can support the development of the system.

- The proposed system can be built using widely available technologies such as Python (Flask/Django), JavaScript (React/Angular), and SQL/NoSQL databases.
- Recommendation algorithms like content-based filtering, collaborative filtering, and hybrid models are well-supported by libraries such as Scikit-learn, TensorFlow, and PyTorch.
- Integration of APIs for location services (e.g., Google Maps API) and notifications is technically achievable.
- Hosting and deployment can be done using platforms like AWS, Heroku, or Firebase.
- Hence, the project is **technically feasible**.

2. Economic Feasibility

Economic feasibility examines whether the project is financially viable based on the cost-benefit analysis.

- The system can be developed with low to moderate investment using open-source technologies.
- Infrastructure costs can be minimized using cloud services with pay-as-you-go pricing models.
- The system has potential commercial value if monetized through promotions, premium features, or event partnerships.
- The long-term benefits such as increased user engagement and event attendance outweigh the initial development cost.
- Therefore, the project is **economically feasible**.

3. Operational Feasibility

Operational feasibility focuses on whether the system can function effectively in a real-world environment and meet user needs.

- The system addresses a real problem: users face difficulty in discovering relevant events.
- It offers practical features like personalized recommendations, location filtering, and timely notifications.
- The user interface can be made intuitive for all age groups with minimal learning curve.
- System maintenance and updates can be managed efficiently.
- Thus, the system is **operationally feasible**.

4. Legal Feasibility

- Legal feasibility ensures that the system complies with legal and ethical standards.
- The system must comply with data protection laws like GDPR or local IT regulations.
- User data will be securely stored and handled with proper consent and privacy policies.

- No legal barriers exist in implementing location services, user tracking (with consent), or feedback collection.
- Therefore, the system is **legally feasible**.

5. Schedule Feasibility

Schedule feasibility evaluates whether the project can be completed within the required timeframe.

- With a well-defined plan, the system can be developed in phases (requirements gathering, design, implementation, testing, deployment).
- Agile methodology can be followed to ensure timely delivery and iterations.
- Depending on the team size and complexity, the core features can be implemented within 3–6 months.
- Hence, the project is **schedule feasible**.

Introduction of Python

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++ or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library

What is Python

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do

- Python can be used on a server to create web applications.

- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.

Machine Learning

Machine learning is a subfield of artificial intelligence (AI). The goal of machine learning generally is to understand the structure of data and fit that data into models that can be understood and utilized by people. Although machine learning is a field within computer science, it differs from traditional computational approaches. In traditional computing, algorithms are sets of explicitly programmed instructions used by computers to calculate or problem solve. Machine learning algorithms instead allow for computers to train on data inputs and use statistical analysis in order to output values that fall within a specific range. Because of this, machine learning facilitates computers in building models from sample data in order to automate decision-making processes based on data inputs. Any technology user today has benefitted from machine learning. Facial recognition technology allows social media platforms to help users tag and share photos of friends. Optical character recognition (OCR) technology converts images of text into movable type. Recommendation engines, powered by machine learning, suggest what movies or television shows to watch next based on user preferences. Self-driving cars that rely on machine learning to navigate may soon be available to consumers. Machine learning is a continuously developing field. Because of this, there are some considerations to keep in mind as you work with machine learning methodologies, or analyze the impact of machine learning processes. In this tutorial, we'll look into the common machine learning methods of supervised and unsupervised learning, and common algorithmic approaches in machine learning, including the k-nearest neighbor algorithm, decision tree learning, and deep learning. We'll explore which programming languages are most used in machine learning, providing you with some of the positive and negative attributes of each. Additionally, we'll discuss biases that are perpetuated by machine learning algorithms, and consider what can be kept in mind to prevent these biases when building algorithms.

Machine Learning Methods

In machine learning, tasks are generally classified into broad categories. These categories are based on how learning is received or how feedback on the learning is given to the system developed.

Two of the most widely adopted machine learning methods are supervised learning which trains algorithms based on example input and output data that is labeled by humans, and unsupervised learning which provides the algorithm with no labeled data in order to allow it to find structure within its input data. Let's explore these methods in more detail.

Supervised Learning

In supervised learning, the computer is provided with example inputs that are labeled with their desired outputs. The purpose of this method is for the algorithm to be able to "learn" by comparing its actual output with the "taught" outputs to find errors, and modify the model accordingly. Supervised learning therefore uses patterns to predict label values on additional unlabeled data.

For example, with supervised learning, an algorithm may be fed data with images of sharks labeled as fish and images of oceans labeled as water. By being trained on this data, the supervised learning algorithm should be able to later identify unlabeled shark images as fish and unlabeled ocean images as water.

A common use case of supervised learning is to use historical data to predict statistically likely future events. It may use historical stock market information to anticipate upcoming fluctuations, or be employed to filter out spam emails. In supervised learning, tagged photos of dogs can be used as input data to classify untagged photos of dogs.

Unsupervised Learning

In unsupervised learning, data is unlabeled, so the learning algorithm is left to find commonalities among its input data. As unlabeled data are more abundant than labeled data, machine learning methods that facilitate unsupervised learning are particularly valuable.

The goal of unsupervised learning may be as straightforward as discovering hidden patterns within a dataset, but it may also have a goal of feature learning, which allows the computational machine to automatically discover the representations that are needed to classify raw data.

Unsupervised learning is commonly used for transactional data. You may have a large dataset of customers and their purchases, but as a human you will likely not be able to make sense of what similar attributes can be drawn from customer profiles and their types of purchases.

Introduction to Collaborative Filtering

Collaborative Filtering is a recommendation technique that makes automatic predictions (filtering) about the interests of a user by collecting preferences from many users (collaborating).

The fundamental assumption behind collaborative filtering is:

"If two users agree on one issue, they are likely to agree on other issues as well."

For example, if User A and User B both liked "The Matrix" and "Inception," and User A also liked "Interstellar," then the system might recommend "Interstellar" to User B.

Key Characteristics of Collaborative Filtering:

- Does not require item metadata (such as genre or description).
- Uses only user-item interaction data (such as ratings, clicks, or purchases).
- Can uncover hidden relationships between users and items.
- Works well in many domains, including movies, music, e-commerce, and news recommendations.

Challenges of Collaborative Filtering

- **Cold Start Problem:** New users or items lack sufficient data to generate recommendations.
- **Data Sparsity:** If user-item interactions are sparse, finding meaningful patterns can be difficult.

Applications

- **Streaming Services:** Netflix, Spotify suggest content based on user behavior.
- **E-commerce:** Amazon, eBay recommend products based on purchase history.
- **Social Media:** Facebook, Twitter suggest friends or content based on interactions.

Collaborative filtering is a powerful technique for personalized recommendations, leveraging user interactions to predict preferences. It is widely used in various domains such as e-commerce, streaming platforms, and social media. However, challenges like cold start, data sparsity, and scalability must be addressed to enhance its effectiveness.

CHAPTER-4
SOFTWARE AND HARDWARE
REQUIREMENT

4. SOFTWARE AND HARDWARE REQUIREMENTS

SOFTWARE REQUIREMENTS:

- Programming Technology : Python,Streamlit
- Operating System : Linux/Windows/macOS
- Frontend Development : CSS
- Operating System : Windows 8/10/11

HARDWARE REQUIREMENTS:

- Processor : Dual Core/I3/I5/I7
- RAM : 4GB(MIN)
- Hard Disk : 100GB or above

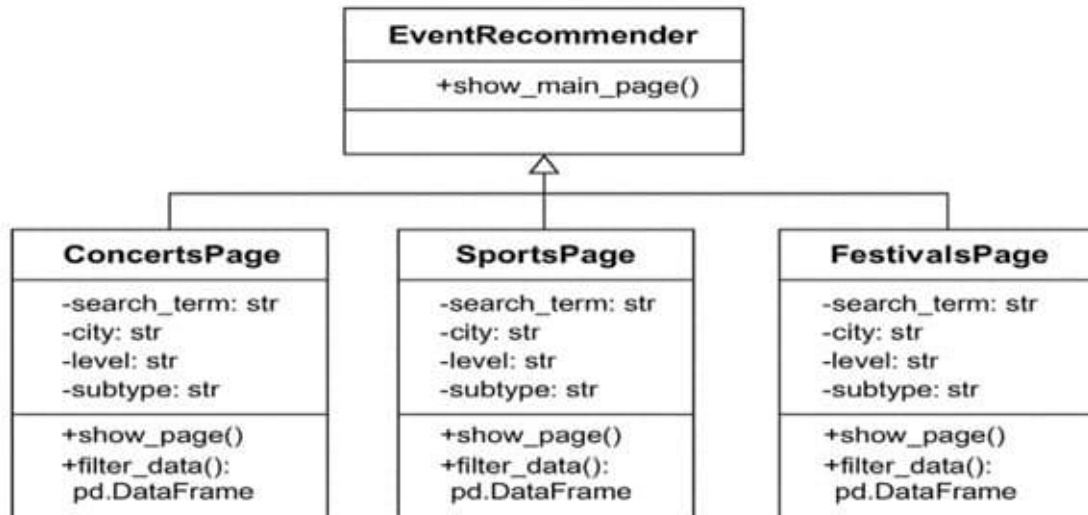
CHAPTER- 5

SYSTEM DESIGN

5. SOFTWARE DESIGN

5.1 DATA FLOW DIAGRAMS

- Class Diagram

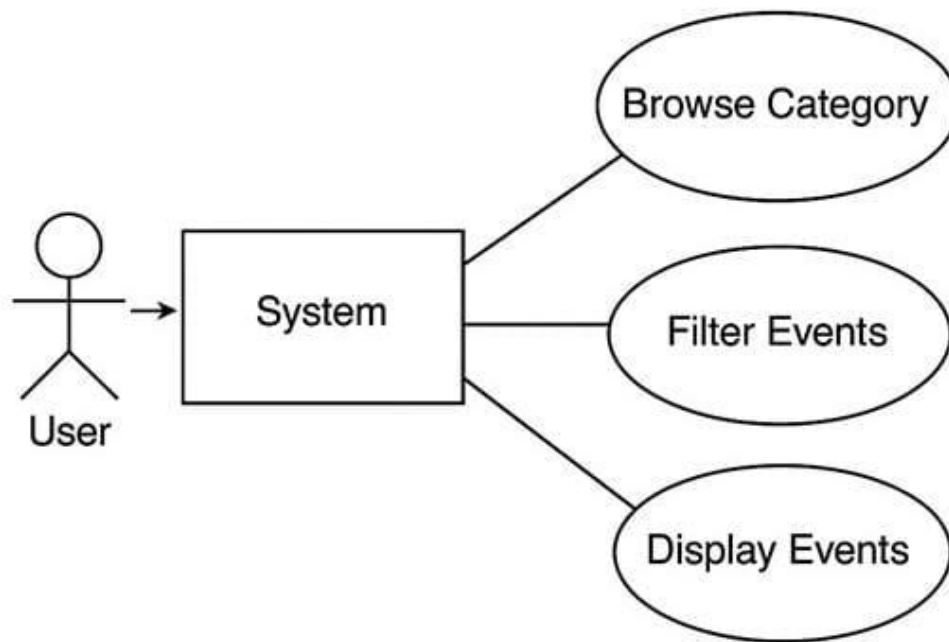


5.1.1 Class Diagram

- User – Stores user details like name and preferred city, level, and event subtype; interacts with the system to explore events.
- FilterManager – Handles applying user-selected filters (city, level, subtype) to event datasets for personalized viewing.
- Event – Represents an individual event with attributes like name, date, city, level, subtype, and popularity.
- Event Suggestion Engine – Filters and ranks events based on popularity and user-selected criteria.
- UI – Collects user input through dropdowns and buttons, and displays matching events in a styled card layout.

5.2 UML DIAGRAMS

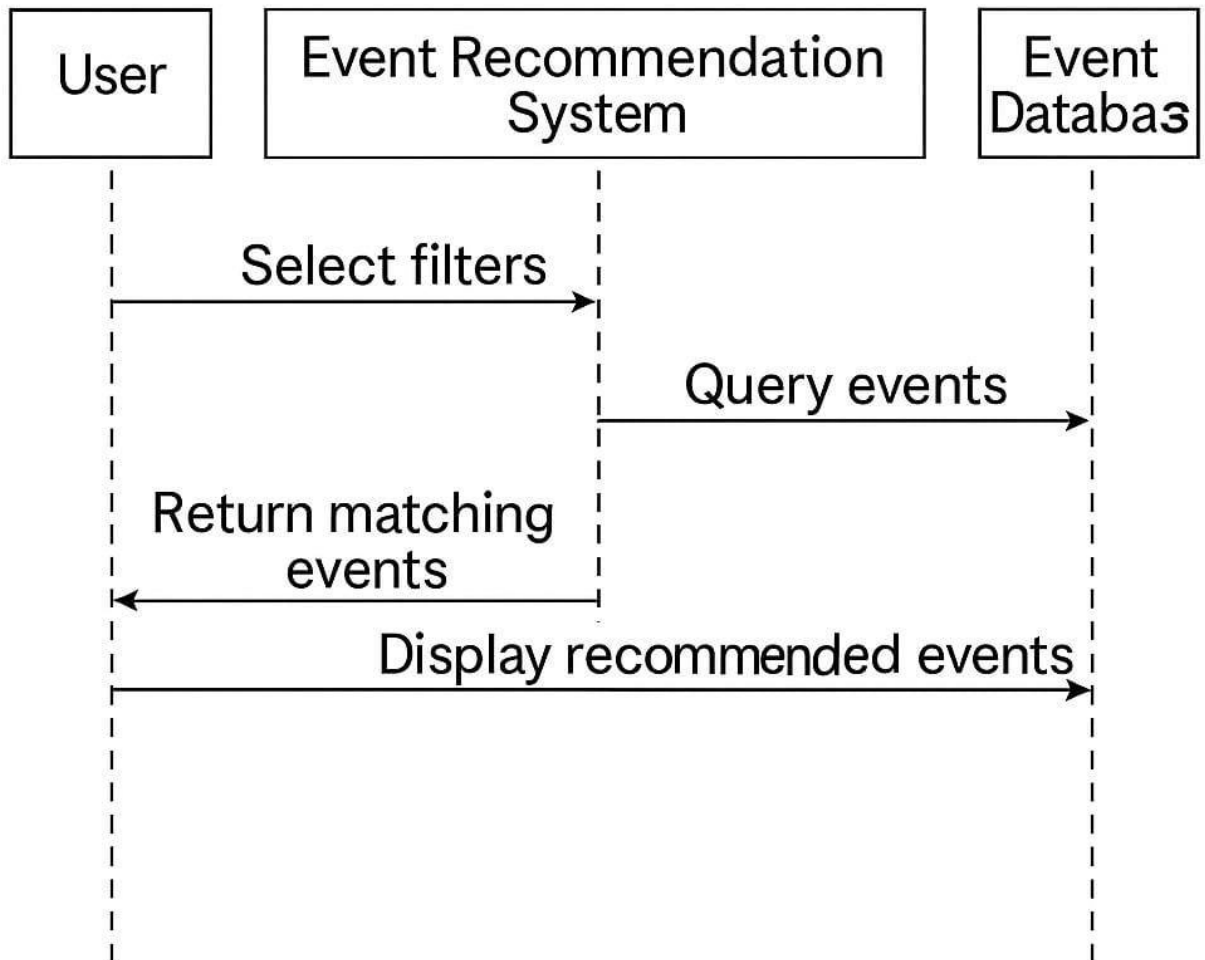
- Use Case Diagram



5.2.1 Use Case Diagram

- Provide Preferences & Location – User selects filters like city, event level, and subtype (e.g., Jazz, Cricket).
- Store Preferences – System captures user-selected filters (if session-based or extended for login features).
- Retrieve Preferences – System retrieves stored or currently selected filters to apply to event data.
- Fetch Event Data – System loads events from CSV files (e.g., concerts.csv, sports.csv) into memory.
- Suggest Events – System filters and ranks events based on user preferences and popularity.
- View Recommended Events – User sees a grid of event cards tailored to their selections

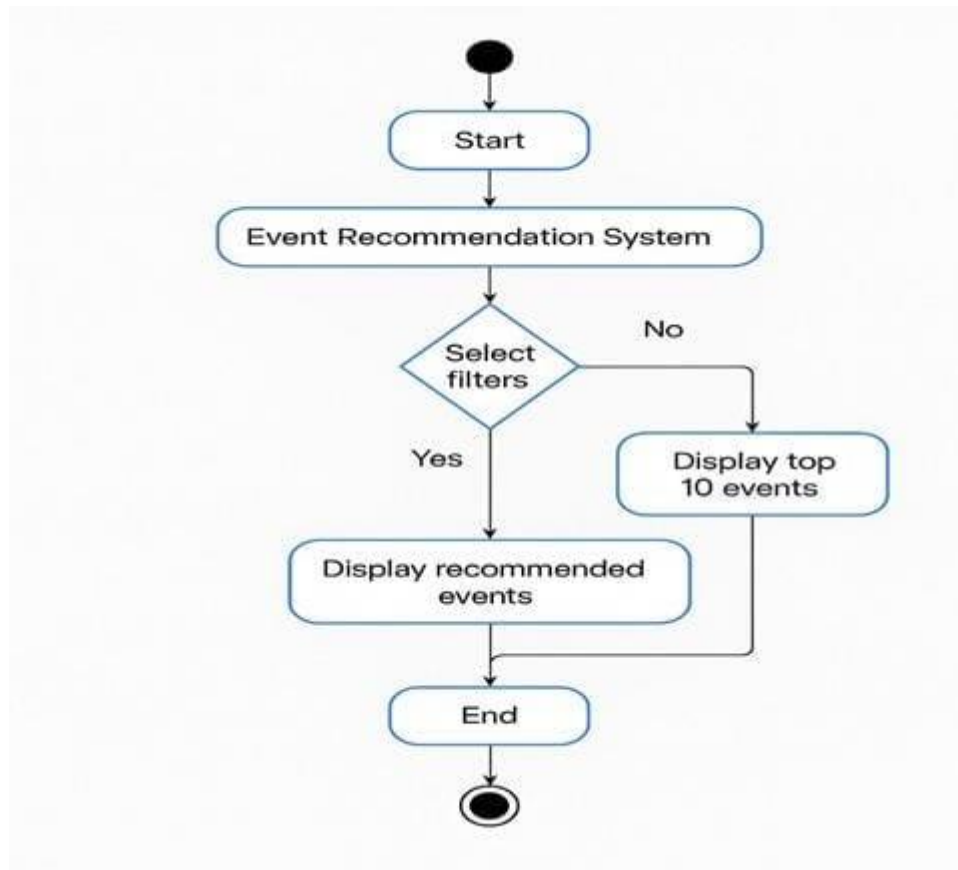
- Sequence Diagram



5.2.2 Sequence Diagram

- User enters preferences and location (e.g., city, event level, subtype) into the system UI.
- Preferences are stored in the FilterManager or session state.
- User requests event recommendations, triggering the filtering process.
- The system retrieves user preferences and fetches relevant event data from the CSV/database.
- The EventSuggestionEngine filters and ranks events based on the preferences.
- The system displays personalized event cards (e.g., concerts or sports) to the user in a grid layout.
- If the user selects an event, detailed event information is retrieved and displayed, including location, date, and popularity.

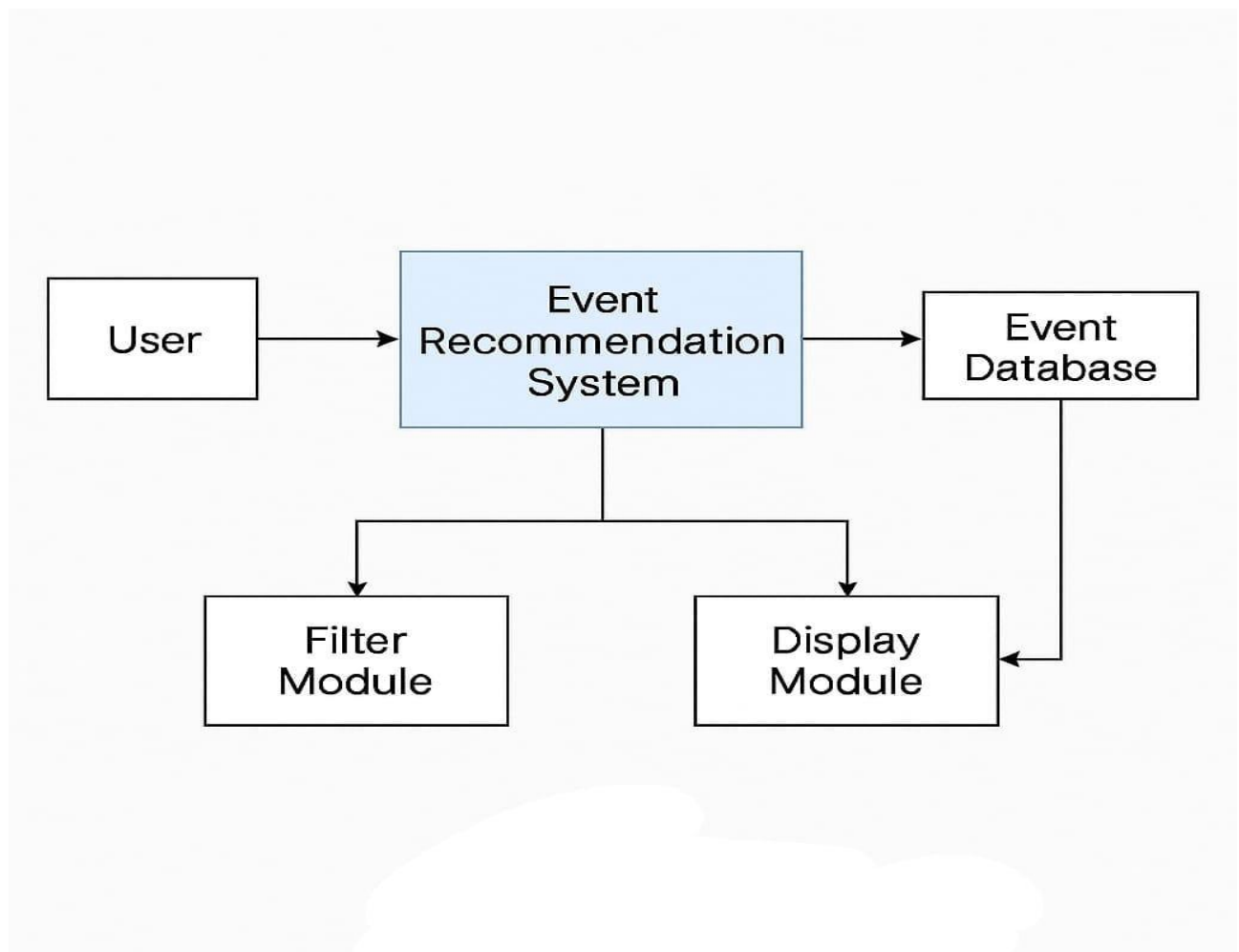
- **Activity Diagram**



5.2.3 Activity Diagram

- User opens the app and provides event preferences (e.g., city, level, event type).
- Preferences are stored temporarily (session) or persistently (if login system is added).
- The system loads event data (e.g., from CSV or database).
- The system analyzes user preferences and filters relevant events.
- Recommended events are displayed to the user in a grid layout.
- If the user selects an event, detailed event info is shown (date, location, subtype, popularity).
- If not, the user can modify preferences to get a new set of recommendations.

5.3 SYSTEM ARCHITECTURE



5.3.1 System Architecture

- User (Tourist) – The person using the application, providing input via filters like city, level, and event type.
- User Interface (UI) – The Streamlit-based frontend where users interact with the system, select preferences, and view recommended events.
- Recommendation Engine – The core logic that filters, ranks, and recommends events based on user-selected criteria and popularity scores.
- Data Source / Database – Static CSV files (e.g., concerts.csv, sports.csv) that store event information including name, city, date, subtype, and popularity.
- Filtering, Sorting, Matching Engine – Processes that match user preferences to available events and sort them by relevance and popularity.
- Session-Based Feedback – Captures current user filters (e.g., selected city or level) to personalize suggestions in real time.

CHAPTER - 6

CODING AND IMPLEMENTATION

6. CODING AND IMPLEMENTATION

6.1 METHODOLOGY

Developing an event recommendation system involves a structured methodology that includes data collection, model development, and system integration. The implementation follows key stages to ensure efficiency and accuracy.

Methodology for Coding

A. Requirement Analysis

- Identified core functionalities: event filtering, category navigation, data display, and UI responsiveness.
- Determined required filters: city, level, subtype.

B. Data Preparation

- Collected and structured event data into CSV files (concerts.csv, sports.csv).
- Ensured fields like name, date, city, level, subtype, and popularity were standardized.

C. UI Development with Streamlit

- Built an intuitive interface using Streamlit widgets like selectbox, multiselect, and st.markdown for layout.
- Used CSS and HTML within Streamlit for custom-styled event cards.

□ Implementation Approach

A. System Architecture & Integration

- Develop a modular architecture that cleanly separates event data handling, filtering logic, and user interface components.
- Integrate real-time APIs for live event data, ticket availability, geolocation, and crowd level estimation.
- Enable multi-source data integration from platforms like Eventbrite, Ticketmaster, and local venues.

B. Testing and Optimization

- Implement unit testing for all filter and sorting functions to validate event recommendation logic.
- Conduct integration testing to ensure smooth interaction between the UI, data loader, and recommendation modules.
- Optimize query performance and rendering for faster page loads and improved scalability with large datasets.

C. Deployment & User Feedback

- Deploy the complete system using cloud platforms such as AWS, Heroku, or Streamlit Cloud for broader accessibility.
- Incorporate user login to store preferences for persistent, personalized recommendations.
- Collect user interaction data and use it to train adaptive or machine learning-based recommendation models.

6.2 SAMPLE CODE

Main.py

```
import streamlit as st
```

```
st.set_page_config(page_title="Event Recommender", layout="centered")
```

```
st.markdown("""
```

```
<style>
```

```
.btn-container {
```

```
    display: flex;
```

```
    justify-content: space-around;
```

```
    margin-top: 60px;
```

```
}
```

```
.btn {
```

```

display: inline-block;

padding: 20px 40px;

background: linear-gradient(135deg, #4c6ef5, #15aabf);

color: white;

border-radius: 20px;

font-size: 20px;

font-weight: bold;

text-decoration: none;

transition: all 0.3s ease-in-out;

box-shadow: 0 8px 16px rgba(0,0,0,0.2);
}

.btn:hover {

background: linear-gradient(135deg, #15aabf, #4c6ef5);

transform: translateY(-5px);

}

</style>

"""', unsafe_allow_html=True)

st.title("🎉 Welcome to the Event Recommendation System")

st.markdown("""

<div class="btn-container">

<a href="/Concerts" class="btn"> 🎵 Concerts</a>

<a href="/Sports" class="btn"> 🏟️ Sports</a>

```

```
<a href="/Festivals" class="btn">🏰 Festivals</a>
```

```
</div>
```

```
""", unsafe_allow_html=True)
```

concerts.py

```
import streamlit as st
```

```
import pandas as pd
```

```
import streamlit.components.v1 as components
```

```
st.set_page_config(page_title="Concert Recommendations", layout="wide")
```

```
st.title("🎵 Concert Recommendations")
```

```
try:
```

```
    df = pd.read_csv("concerts.csv")
```

```
except FileNotFoundError:
```

```
    st.error("❌ 'concerts.csv' not found.")
```

```
    st.stop()
```

```
# Dropdown filters
```

```
cities = sorted(df["city"].dropna().unique().tolist())
```

```
levels = sorted(df["level"].dropna().unique().tolist())
```

```
subtypes = sorted(df["subtype"].dropna().unique().tolist())
```

```
city = st.selectbox("Select City", options=[""] + cities)
```

```
level = st.selectbox("Select Level", options=[""] + levels)
```

```
subtype = st.selectbox("Select Subtype", options=[""] + subtypes)
```

```
# Filter logic
```

```

filtered = df.copy()

if city:

    filtered = filtered[filtered["city"] == city]

if level:

    filtered = filtered[filtered["level"] == level]

if subtype:

    filtered = filtered[filtered["subtype"] == subtype]

# Sort by popularity

filtered = filtered.sort_values(by="popularity", ascending=False)

# Determine whether to show top 10 or filtered results

if not city and not level and not subtype:

    display_df = df.sort_values(by="popularity", ascending=False).head(10)

    st.subheader("🔥 Top 10 Trending Concerts")

else:

    display_df = filtered

    st.subheader("🎯 Recommended Concerts")

if display_df.empty:

    st.warning("No concerts found matching the selected filters.")

else:

    # Grid layout with 3 per row

    cards_html = ""

    <style>

    .grid-container {

        display: flex;

```

```
flex-wrap: wrap;

gap: 20px;

justify-content: flex-start;

padding-top: 10px;
}

.event-card {

background-color: #ffffff;

border-radius: 15px;

box-shadow: 0 4px 12px rgba(0,0,0,0.1);

padding: 20px;

width: calc(33% - 20px);

box-sizing: border-box;

transition: transform 0.2s ease, box-shadow 0.3s ease;
}

.event-card:hover {

transform: translateY(-5px);

box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}

.event-title {

font-size: 18px;

font-weight: bold;

color: #3f51b5;
}

.event-info {
```



```

font-size: 14px;

color: #333;

margin-top: 6px;

}

</style>

<div class="grid-container">

    """

    for _, row in display_df.iterrows():

        cards_html += f"""

        <div class="event-card">

            <div class="event-title">{row['name']}</div>

            <div class="event-info">📅 {row['date']}</div>

            <div class="event-info">📍 {row['city']}</div>

            <div class="event-info">🏷️ {row['subtype']}</div>

            <div class="event-info">🌐 {row['level']}</div>

            <div class="event-info">🔥 Popularity: {row['popularity']}</div>

        </div>

        """

        cards_html += "</div>"

    components.html(cards_html, height=800, scrolling=True)

```

Sports.py

```

import streamlit as st

import pandas as pd

import streamlit.components.v1 as components

```

```

st.set_page_config(page_title="Sports Recommendations", layout="wide")

st.title("🏆 Sports Event Recommendations")


# Load CSV

try:

    df = pd.read_csv("sports.csv")

except FileNotFoundError:

    st.error("❌ 'sports_events.csv' not found.")

    st.stop()


# Filters

cities = sorted(df["city"].dropna().unique().tolist())

levels = sorted(df["level"].dropna().unique().tolist())

main_types = ["Indoor", "Outdoor"]


city = st.selectbox("Select City", options=[""] + cities)

level = st.selectbox("Select Level", options=[""] + levels)

main_type = st.radio("Select Event Type", options=main_types, horizontal=True)


# Filter subtypes based on Indoor/Outdoor

if main_type == "Indoor":

    subtypes = sorted(df[df["subtype"].str.lower().str.contains("indoor")]["subtype"].unique())

else:

```

```

subtypes = sorted(df[df["subtype"].str.lower().str.contains("outdoor")]["subtype"].unique())

# Remove the "Indoor" or "Outdoor" part from subtype for display

subtypes = [subtype.replace("Indoor ", "").replace("Outdoor ", "") for subtype in subtypes]

subtype = st.selectbox("Select Sport", options=[""] + subtypes)

# Apply Filters

filtered = df.copy()

if city:

    filtered = filtered[filtered["city"] == city]

if level:

    filtered = filtered[filtered["level"] == level]

if subtype:

    filtered = filtered[filtered["subtype"].str.contains(subtype, case=False)]

filtered = filtered.sort_values(by="popularity", ascending=False)

# Display Results

if not city and not level and not subtype:

    display_df = df.sort_values(by="popularity", ascending=False).head(10)

    st.subheader("🔥 Top 10 Trending Sports Events")

else:

    display_df = filtered

```

```
st.subheader("🎯 Recommended Sports Events")
```

```
if display_df.empty:
```

```
    st.warning("No events found matching the selected filters.")
```

```
else:
```

```
    # Grid Layout
```

```
    cards_html = ""
```

```
    <style>
```

```
    .grid-container {
```

```
        display: flex;
```

```
        flex-wrap: wrap;
```

```
        gap: 20px;
```

```
        justify-content: flex-start;
```

```
        padding-top: 10px;
```

```
    }
```

```
    .event-card {
```

```
        background-color: #ffffff;
```

```
        border-radius: 15px;
```

```
        box-shadow: 0 4px 12px rgba(0,0,0,0.1);
```

```
        padding: 20px;
```

```
        width: calc(33% - 20px);
```

```
        box-sizing: border-box;
```

```
        transition: transform 0.2s ease, box-shadow 0.3s ease;
```

```
    }
```

```
.event-card:hover {

    transform: translateY(-5px);

    box-shadow: 0 8px 20px rgba(0,0,0,0.15);

}
```

```
.event-title {

    font-size: 18px;

    font-weight: bold;

    color: #2e7d32;

}
```

```
.event-info {

    font-size: 14px;

    color: #333;

    margin-top: 6px;

}
```

```
</style>
```

```
<div class="grid-container">
```

```
""
```

```
for _, row in display_df.iterrows():
```

```
    # Display only the sport name without "Indoor" or "Outdoor"
```

```
    event_subtype = row['subtype'].replace("Indoor ", "").replace("Outdoor ", "")
```

```
    cards_html += f"""
```

```
        <div class="event-card">
```

```
            <div class="event-title">{row['name']}</div>
```

```
<div class="event-info"> 📅 {row['date']}</div>
```

```
<div class="event-info"> 📍 {row['city']}</div>
```

```
<div class="event-info"> 🏷️ {event_subtype}</div>
```

```
<div class="event-info"> 🌐 {row['level']}</div>
```

```
<div class="event-info"> 🔥 Popularity: {row['popularity']}</div>
```

```
</div>
```

```
""
```

```
cards_html += "</div>"
```

```
components.html(cards_html, height=800, scrolling=True)
```

Festivals.py

```
import streamlit as st
```

```
import pandas as pd
```

```
import streamlit.components.v1 as components
```

```
st.set_page_config(page_title="Festival Recommendations", layout="wide")
```

```
st.title(" 🏰 Festival Event Recommendations")
```

```
# Load CSV
```

```
try:
```

```
    df = pd.read_csv("festivals.csv")
```

```
except FileNotFoundError:
```

```
    st.error(" ❌ 'festivals.csv' not found.")
```

```
st.stop()
```

```
# Filters
```

```
cities = sorted(df["city"].dropna().unique().tolist())
```

```
levels = sorted(df["level"].dropna().unique().tolist())
```

```
types = sorted(df["subtype"].dropna().unique().tolist())
```

```
city = st.selectbox("Select City", options=[""] + cities)
```

```
level = st.selectbox("Select Level", options=[""] + levels)
```

```
subtype = st.selectbox("Select Festival Type", options=[""] + types)
```

```
# Apply Filters
```

```
filtered = df.copy()
```

```
if city:
```

```
    filtered = filtered[filtered["city"] == city]
```

```
if level:
```

```
    filtered = filtered[filtered["level"] == level]
```

```
if subtype:
```

```
    filtered = filtered[filtered["subtype"] == subtype]
```

```
filtered = filtered.sort_values(by="popularity", ascending=False)
```

```
# Display Results
```

```
if not city and not level and not subtype:
```

```
    display_df = df.sort_values(by="popularity", ascending=False).head(10)
```

```
st.subheader("🔥 Top 10 Trending Festivals")
```

```
else:
```

```
display_df = filtered
```

```
st.subheader("🎯 Recommended Festivals")
```

```
if display_df.empty:
```

```
    st.warning("No festivals found matching the selected filters.")
```

```
else:
```

```
    # Grid Layout for Events
```

```
    cards_html = ""
```

```
    <style>
```

```
    .grid-container {
```

```
        display: flex;
```

```
        flex-wrap: wrap;
```

```
        gap: 20px;
```

```
        justify-content: flex-start;
```

```
        padding-top: 10px;
```

```
    }
```

```
    .event-card {
```

```
        background-color: #ffffff;
```

```
        border-radius: 15px;
```

```
        box-shadow: 0 4px 12px rgba(0,0,0,0.1);
```

```
        padding: 20px;
```

```
        width: calc(33% - 20px);
```



```

    box-sizing: border-box;

    transition: transform 0.2s ease, box-shadow 0.3s ease;
}

.event-card:hover {

    transform: translateY(-5px);

    box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}

.event-title {

    font-size: 18px;

    font-weight: bold;

    color: #8e44ad;
}

.event-info {

    font-size: 14px;

    color: #333;

    margin-top: 6px;
}

</style>

<div class="grid-container">

    ""

```

```

for _, row in display_df.iterrows():

    cards_html += f""

    <div class="event-card">

```

```
<div class="event-title">{row['name']}</div>
```

```
<div class="event-info">📅 {row['date']}</div>
```

```
<div class="event-info">📍 {row['city']}</div>
```

```
<div class="event-info">🏷️ {row['subtype']}</div>
```

```
<div class="event-info">🌐 {row['level']}</div>
```

```
<div class="event-info">🔥 Popularity: {row['popularity']}</div>
```

```
</div>
```

```
"""
```

```
cards_html += "</div>"
```

```
components.html(cards_html, height=800, scrolling=True)
```

Contact.py

```
import streamlit as st
```

```
st.set_page_config(page_title="Contact Us", layout="centered")
```

```
st.title("📞 Contact Information")
```

```
st.markdown("Feel free to reach out to us using the details below:")
```

```
# Divider
```

```
st.markdown("----")
```

```
# Contact Info Section
```

```
st.markdown("""
```

```
### 📧 Get in Touch
```

```
- **Email**: ultronmegatron19@gmail.com
```

```
- **Phone**: +91 8374466035
```

```
- **Address**:
```

```
    Event Recommender Inc.
```

```
    CMR Engineering College Medchal-500401.
```

```
""")
```

```
# Optional: Footer or support notice
```

```
st.markdown("---")
```

```
st.info("We typically respond to emails within 24 hours.")
```

6.3 DATA DICTIONARY

Event Data

This dataset forms the core of the system, containing metadata for all listed sports and festival events.

Event ID: A unique identifier for each event.

Name: The title or name of the event (e.g., "City Marathon", "Music Fest 2025").

City: The location where the event is being held.

Venue: The specific address or venue of the event (if available).

Date: The scheduled date of the event in YYYY-MM-DD format.

Subtype: A more specific classification of the event (e.g., "Indoor Football", "Food Festival").

Main Type: General category, either "Sports" or "Festivals".

Level: Indicates the scope of the event — such as "Local", "State", "National", or "International".

Popularity: A numeric value or score representing the trending status of the event, often derived from user activity or event attendance.

Ticket Cost: Estimated cost of entry (optional; applicable mostly for festivals).

Description: Brief details about the event.

Event Image: A reference to the event poster or image URL for display in the UI.

Filter & Preference Settings

This segment defines the dynamic filtering criteria applied by users to narrow down event suggestions.

Selected City: The city chosen from the dropdown to view local events.

Selected Level: The preferred level of event chosen (e.g., "National", "International").

Selected Event Type: "Indoor" or "Outdoor" — determines how subtypes are filtered.

Selected Subtype: Sport or festival subtype selected (e.g., "Basketball", "Music").

Sort Preference: The system sorts events by default using the "popularity" field to surface top-rated events first.

Display & Recommendation Data

Used for organizing and rendering filtered or recommended events in the user interface.

Top Trending Events: Top 10 events globally (based on popularity), shown when no filters are applied.

Recommended Events: List of events matching user-defined filters.

Grid Display Metadata: Style and structure used for rendering cards, including:

Event name, date, city, subtype, level, and popularity icon/score.

CHAPTER – 7

SYSTEM TESTING

7. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

Types of System Testing

- Unit Testing
- Integration Testing
- Functional Testing
- Performance Testing
- Security Testing
- Usability Testing
- Compatibility Testing
- Regression Testing

7.1 SYSTEM TESTCASES

Test Case	Test Data	Expected Result	Actual Result	Status (Pass/Fail)
Launch Application	Click on the "Run" button	Application starts successfully	Application starts successfully	Pass
Dataset Location	Selecting event dataset	Dataset is loaded into the system	Dataset is loaded into the system	Pass
Run the Algorithms	Running recommendation algorithm on dataset	Event recommendations are generated	Event recommendations are generated	Pass

7.2 UNIT TESTING

Test Case id	Module	Test Case	Test Data	Expected Result	Actual Result	Status
UT-01	Data Processing	Load Events Dataset	Sample dataset with events	Dataset is loaded successfully	Dataset is loaded	Pass
UT-02	Data Processing	Clean Dataset	Dataset with missing values	Data is cleaned without errors	Data is cleaned	Pass
UT-03	Recommendation Engine	Generate Recommendations	User preferences and history	Relevant Event suggestions generated	Recommendations generated	Pass

7.3 INTEGRATION TESTING

Test case id	Description	Expected Result	Status
1	Destination Ranking Accuracy	System ranks travel events correctly based on user preferences.	Pass
2	User Preference Update	Users see updated recommendations when preferences change.	Pass
3	Data Retrieval	System fetches events, level, and place data accurately.	Pass

CHAPTER - 8

OUTPUT SCREENS

8. OUTPUT SCREENS

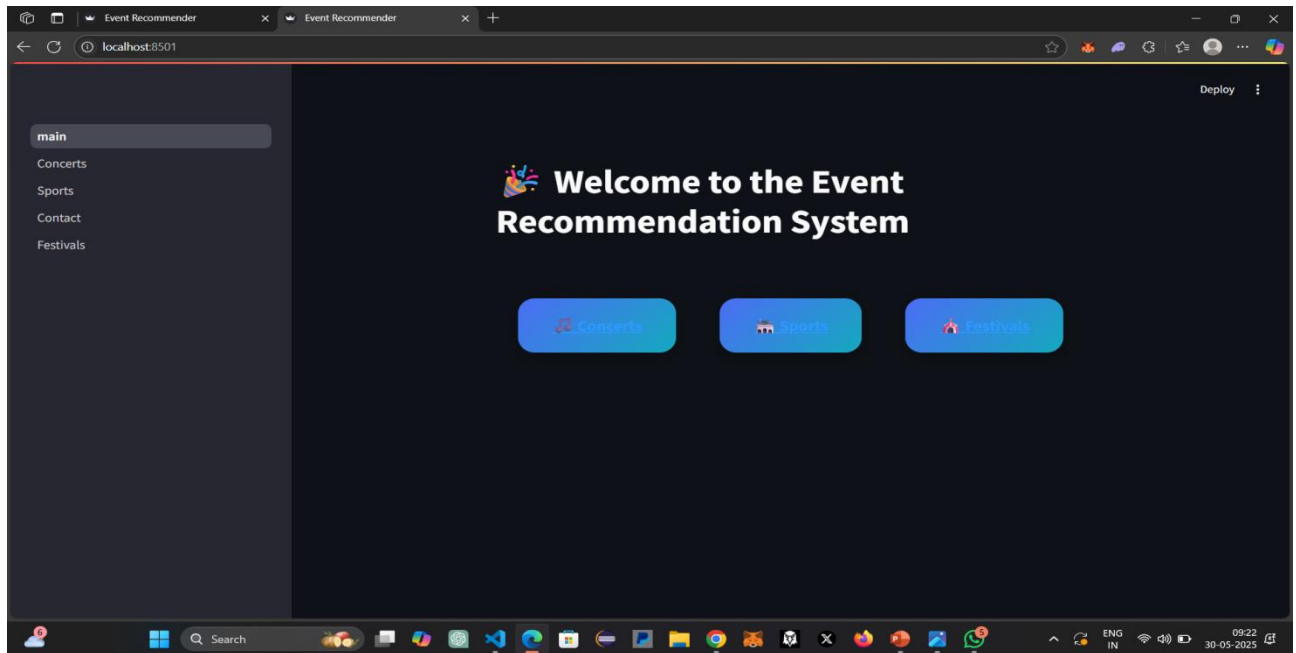


Figure 8.a : Main Page

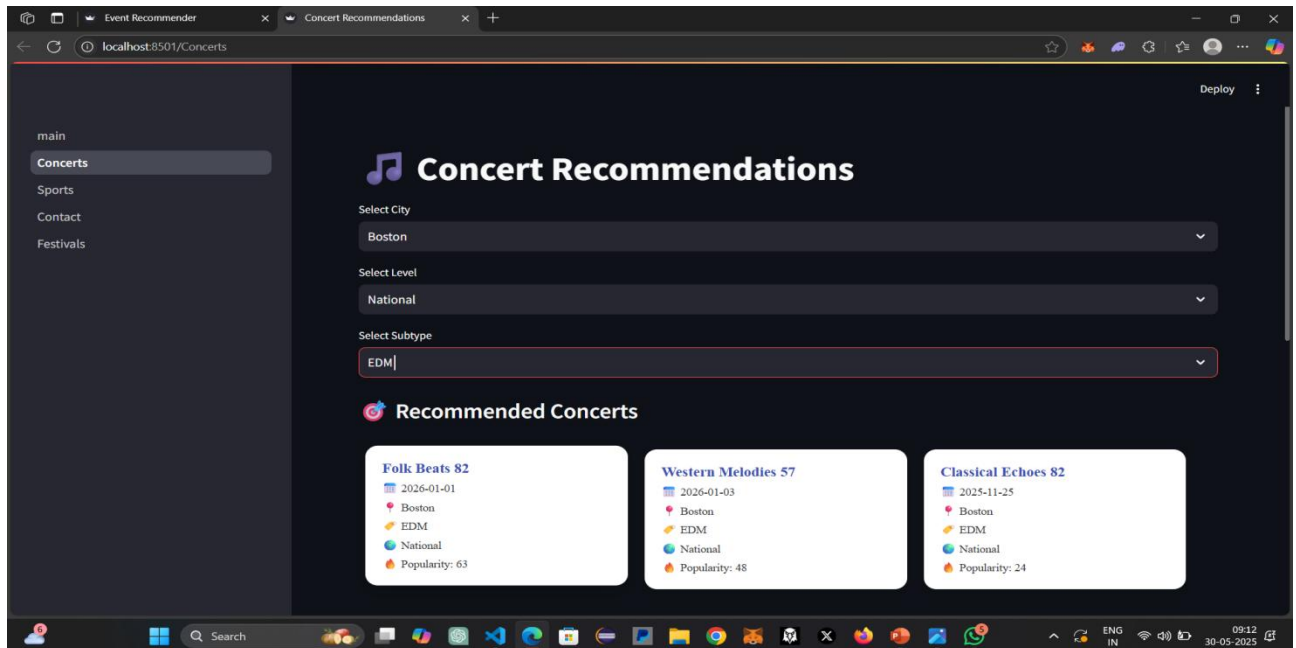


Figure 8.b : Concerts recommendation Page

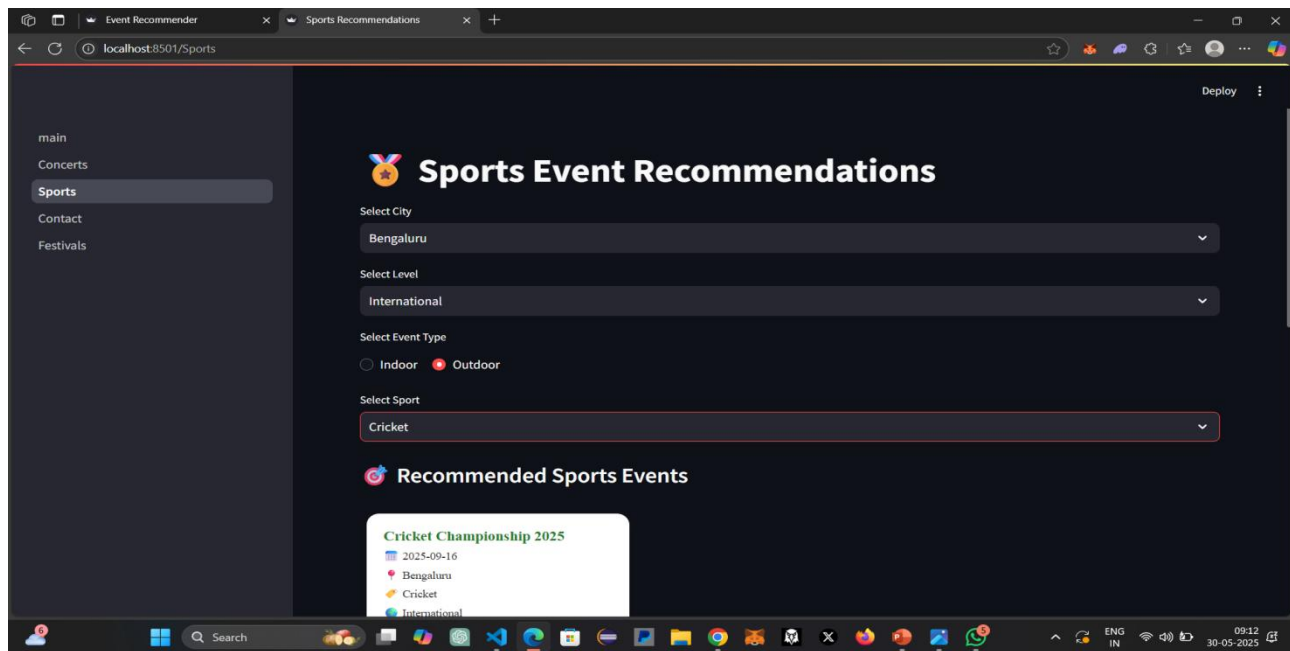


Figure 8.c : Sports recommendation page

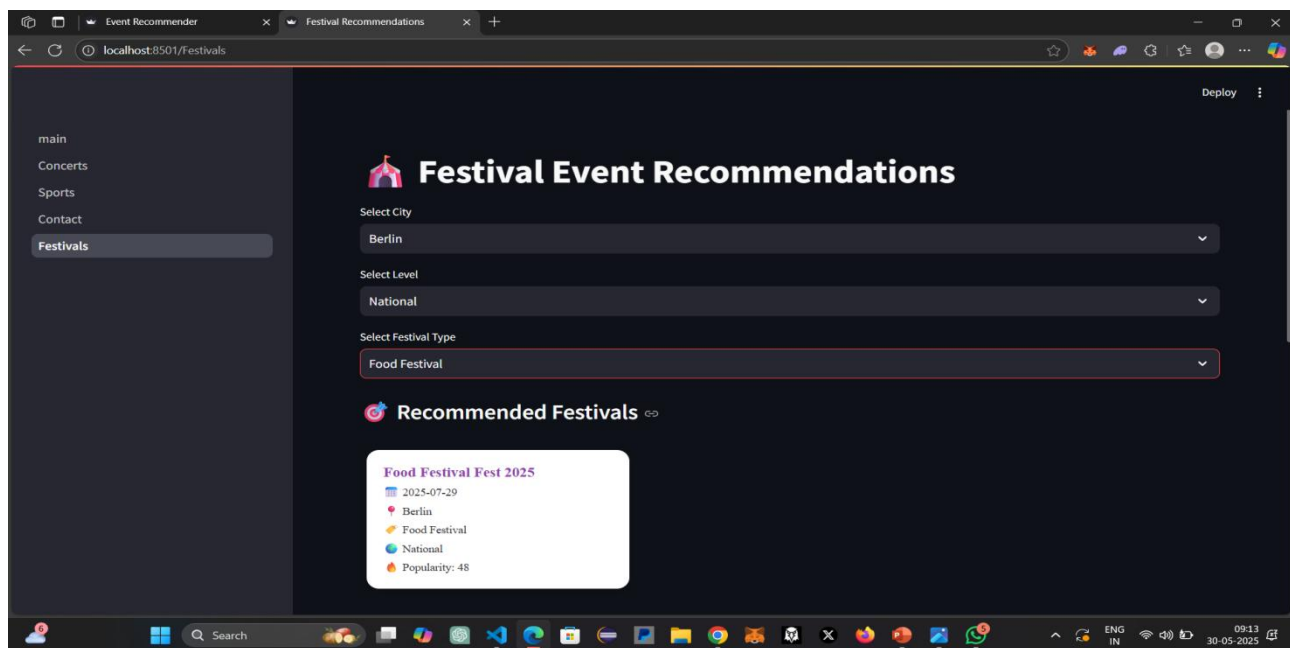


Figure 8.d : Festivals recommendation Page

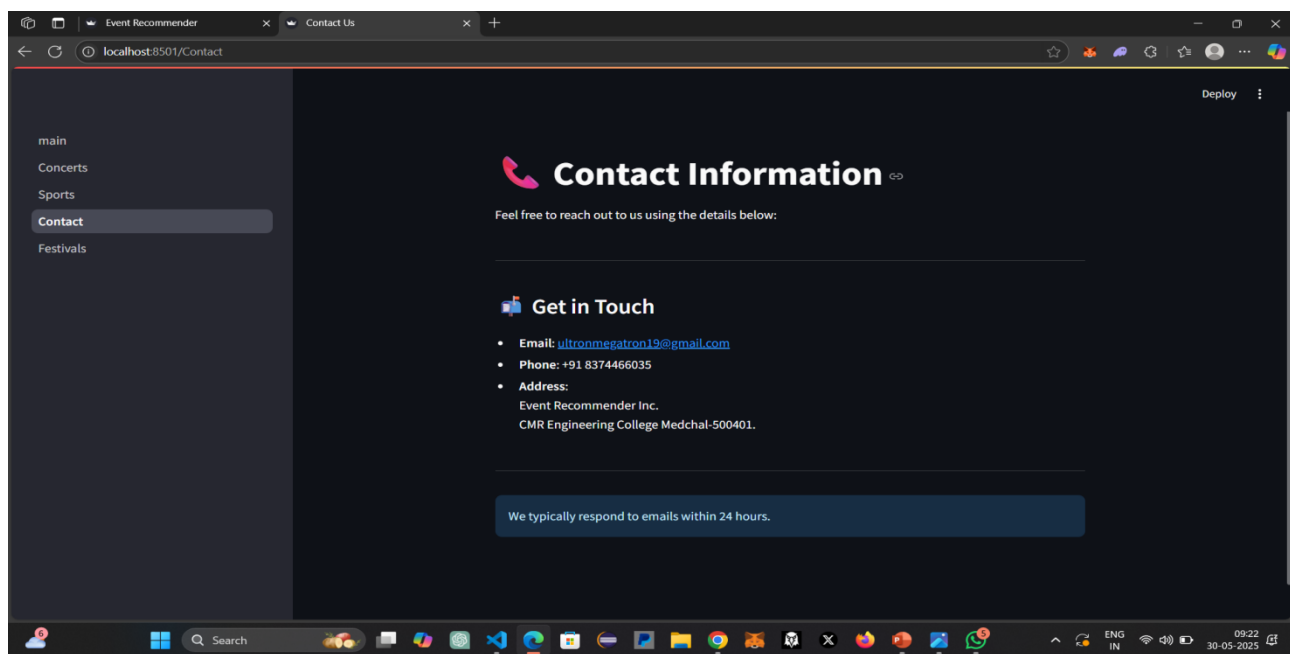


Figure 8.e : Contact Page

CHAPTER - 9
CONCLUSION

9. CONCLUSION

The Event Recommendation System effectively demonstrates a user-friendly, data-driven approach to discovering and filtering both sports and festival events based on user preferences. Designed using Streamlit, the application dynamically processes event data from structured CSV files and presents relevant recommendations through a clean and interactive web interface. With features such as city, level, event type, and popularity filters, users can quickly navigate a wide array of events tailored to their interests. The use of custom HTML and CSS card layouts enhances the visual presentation, making the system not only functional but also engaging. This project showcases how lightweight, real-time recommendation systems can be built without complex user profiles or databases, making it ideal for local discovery platforms, event promotion portals, or city-based cultural applications. While simple, it is also extensible — future enhancements like user login systems, personalized recommendations, map integrations, or live data APIs can make the platform even more powerful and personalized. In conclusion, the system provides a solid foundation for scalable, location-aware event discovery, blending clean design, intuitive interaction, and practical filtering logic.

CHAPTER – 10

FUTURE ENHANCEMENT

10. FUTURE ENHANCEMENT

The Event Recommendation System can be enhanced by integrating a dynamic database to replace static CSVs, enabling real-time updates, multi-user support, and improved scalability. Incorporating machine learning algorithms for personalized recommendations based on user preferences and behavior would significantly improve accuracy and user satisfaction. Features such as user authentication, profile management, and notification services (email or SMS reminders) would enhance user engagement and retention. Additionally, integrating third-party event APIs like Eventbrite or Ticketmaster can ensure that event listings remain current and comprehensive.

Making the system mobile-friendly or developing it as a Progressive Web App (PWA) would provide better accessibility across devices. An admin panel for managing event data and analyzing user interaction metrics could support more efficient system maintenance and data-driven improvements. Implementing feedback and rating mechanisms would help refine recommendations and build trust among users. Future updates could also include multilingual support and location-based suggestions, making the system more inclusive and context-aware. These enhancements would transform the application into a smart, responsive, and scalable event discovery platform.

11. REFERENCES

- [1] Xiong, L., et al. (2017). *Temporal Collaborative Filtering with Graph Convolutional Neural Networks*. <https://arxiv.org/pdf/2010.06425>
- [2] Li, X., et al. (2019). *Event Recommendation using Collaborative Filtering and Event Embeddings*. <https://iarjset.com/wp-content/uploads/2024/12/IARJSET.2024.111124>
- [3] Li, X., et al. (2020). *Deep Collaborative Filtering for Event Recommendation using Convolutional Neural Networks*. <https://iarjset.com/wp-content/uploads/2024/12/IARJSET.2024.111124>
- [4] Kim, D., et al. (2018). *Scalable Collaborative Filtering for Event Recommendation using Distributed Tensor Factorization*.
<https://iarjset.com/wp-content/uploads/2024/12/IARJSET.2024.111124.pdf>

11.1 TEXTBOOKS

1. Recommender Systems: The Textbook – Charu C. Aggarwal
2. Data Science for Business – Foster Provost, Tom Fawcett
3. Mastering Machine Learning with Python in Six Steps – Manohar Swamy Nathan
4. Practical Recommender Systems – Kim Falk
5. Web Scraping with Python: Collecting Data from the Modern Web – Ryan Mitchell

11.2 WEBSITES

1. Chatgpt – <https://www.Chatgpt.com> (Datasets and ML models for recommendations)
2. Streamlit – App Framework for Machine Learning and Data Science  <https://streamlit.io/>
3. Python Official Website  <https://www.python.org/>
4. Pandas – Python Data Analysis Library  <https://pandas.pydata.org/>